Now that we've told PHP we want to use `$txt` from global scope, the code executes without any issues. All variables declared in global scope will have their names added as keys to the special `$GLOBALS` array and their data can be accessed accordingly. If the variable already exists, it will have its value replaced and herein lies one of the danger with global variables. Another thing to consider with using `global` inside a function is that the global variable will be created if it doesn't already exist:

```
function testGlobalVars(): void
{
    global $txt;
    $txt = 'Watch me stamp all over global data...';
}

var_dump($txt);
testGlobalVars();
var_dump($txt);
```

```
Output:
Warning: Undefined variable $txt in /usr/src/myapp/scope8.php on line 9
NULL
string(38) "Watch me stamp all over global data..."
```

As we can see, by declaring `$txt` as `global` in the local scope of `testGlobalVars()`, we have now added `$txt` to global scope. Exactly the same can be achieved by writing `$GLOBALS['txt'] = ...` in the body of `testGlobalVars()`.

```
function testSuperglobal(): void
{
    $GLOBALS['foo'] = 'foo';
}

testSuperglobal();
var_dump($GLOBALS['foo']);
```

```
Output:
string(3) "foo"
```

Yet another danger of global. Hopefully you're beginning to see why global data should be treated with extreme caution.